17. E2E 테스트 실행 및 검증 과정 분석

개요

Omni의 E2E 테스트 실행 과정은 네트워크 배포부터 종합적인 검증까지 여러 단계로 구성됩니다. 이 시스템은 자동화된 테스트 실행, 실시간 모니터링, 장애 시뮬레이션, 그리고 포괄적인 검증 로직을 통해 Omni 프로토콜의 안정성과 신뢰성을 보장합니다.

테스트 실행 단계

1. 전체 E2E 테스트 플로우

```
func E2ETest(ctx context.Context, def Definition, cfg E2ETestConfig) error {
// 1. 네트워크 배포
pingpong, err := Deploy(ctx, def, depCfg)
if err != nil {
return err
}

// 2. 모니터링 시작
stopReceiptMonitor := StartMonitoringReceipts(ctx, def)
stopValidatorUpdates := StartValidatorUpdates(ctx, def)
stopAddingPortals := startAddingMockPortals(ctx, def)
```

```go
// 3. 크로스체인 메시지 전송 시작
msgBatches := []int{3, 2, 1}
msgsErr := StartSendingXMsgs(ctx, def.Testnet.Network, def.Netman(), def.Backend


// 4. 나머지 서비스 시작
if err := StartRemaining(ctx, def.Testnet.Testnet, def.Infra); err != nil {
return err
}


// 5. 초기 대기 및 안정화
if err := Wait(ctx, def.Testnet.Testnet, 5); err != nil {
return err
}


// 6. 장애 시뮬레이션 (Perturbations)
if def.Testnet.HasPerturbations() {
if err := perturb(ctx, def.Testnet); err != nil {
return err
}
}


// 7. 증거 주입 및 슬래싱 테스트
if def.Testnet.Evidence > 0 {
```

```go
valAddr, err := injectEvidence(ctx, def.Testnet)
if err != nil {
return errors.Wrap(err, "inject evidence")
}
if err := awaitSlashed(ctx, def, valAddr); err != nil {
return errors.Wrap(err, "await slashed")
}
}

// 8. 네트워크 업그레이드 완료 대기
if err := ensureNetworkUpgrades(ctx, def); err != nil {
return err
}

// 9. 단위 테스트 실행
if err := Test(ctx, def, TestConfig{Verbose: true}); err != nil {
return err
}

// 10. 최종 검증 및 정리
return finalValidation(ctx, def, pingpong, msgsErr, stopReceiptMonitor, stopVal
}
```

2. 네트워크 배포 과정 (Deploy)

2.1 배포 단계별 실행

```
func Deploy(ctx context.Context, def Definition, cfg DeployConfig) (*pingpong.X

// 1. 설정 및 정리

if err := Setup(ctx, def, cfg); err != nil {

return nil, err

}

if err := CleanInfra(ctx, def); err != nil {

return nil, err

}


// 2. 초기 서비스 시작

if err := StartInitial(ctx, def.Testnet.Testnet, def.Infra); err != nil {

return nil, err

}


// 3. EVM 체인 대기

if err := waitForEVMs(ctx, def.Testnet.EVMChains(), def.Backends()); err != ni

return nil, err

}


// 4. 자금 조달 및 컨트랙트 배포
```

```go
if err = fundAnvil(ctx, def); err != nil {
return nil, errors.Wrap(err, "fund anvil")
}

if err = deployAllCreate3(ctx, def); err != nil {
return nil, errors.Wrap(err, "deploy create3")
}

// 5. 포털 배포
if err := def.Netman().DeployPortals(ctx, genesisValSetID, genesisVals); err !
return nil, errors.Wrap(err, "deploy portals")
}

// 6. 추가 컨트랙트 배포 (병렬)
var eg1 errgroup.Group
eg1.Go(func() error { return maybeDeploySolver(ctx, def) })
eg1.Go(func() error { return maybeDeployL1OmniERC20(ctx, networkFromDef(def), d
eg1.Go(func() error { return maybeDeployNomina(ctx, def) })
if err := eg1.Wait(); err != nil {
return nil, errors.Wrap(err, "deploy solver and erc20 contracts")
}

// 7. 최종 설정 (병렬)
```

```go
var  eg2 errgroup.Group
eg2.Go(func() error { return  initPortalRegistry(ctx, def) })
eg2.Go(func() error { return  allowStagingValidators(ctx, def) })
eg2.Go(func() error { return  maybeSubmitNetworkUpgrades(ctx, def) })
eg2.Go(func() error { return  FundValidatorsForTesting(ctx, def) })
if  err := eg2.Wait();  err != nil {
return  nil, errors.Wrap(err, "deploy other contracts")
}

// 8. 지원 체인 대기 및 PingPong 배포
err = waitForSupportedChains(ctx, def)
if  err != nil {
return  nil, err
}

if  cfg.PingPongN > 0 && def.Testnet.Network != netconf.Mainnet {
pp, err := pingpong.Deploy(ctx, networkFromDef(def), def.Backends())
if  err != nil {
return  nil, errors.Wrap(err, "deploy pingpong")
}
err = pp.StartAllEdges(ctx, cfg.PingPongL, cfg.PingPongP, cfg.PingPongN)
if  err != nil {
return  nil, errors.Wrap(err, "start all edges")
```

```
}
return &pp, nil
}

return nil, nil
}
```

2.2 서비스 시작 순서

1. **Halo 노드들**: 합의 클라이언트 우선 시작
2. **Omni EVM**: Geth 기반 실행 엔진
3. **Anvil 체인들**: 모의 EVM 체인
4. **Relayer**: 크로스체인 메시지 릴레이
5. **Monitor**: 네트워크 모니터링
6. **Solver**: SolverNet 서비스

3. 테스트 설정 및 환경 구성

3.1 설정 파일 생성 (Setup)

```
func Setup(ctx context.Context, def Definition, depCfg DeployConfig) error {
// 1. 디렉토리 정리 및 생성
if err := CleanupDir(ctx, def.Testnet.Dir); err != nil {
return err
}
```

```go
if err := os.MkdirAll(def.Testnet.Dir, os.ModePerm); err != nil {
return errors.Wrap(err, "mkdir")
}

// 2. 기능 플래그 설정
feature.SetGlobals(def.Manifest.FeatureFlags)

// 3. Geth 제네시스 생성
gethGenesis, err := evmgenutil.MakeGenesis(def.Manifest.Network)
if err != nil {
return errors.Wrap(err, "make genesis")
}
if err := geth.WriteAllConfig(def.Testnet, gethGenesis); err != nil {
return err
}

// 4. Halo 제네시스 생성
cosmosGenesis, err := genutil.MakeGenesis(
ctx,
def.Manifest.Network,
time.Now(),
gethGenesis.ToBlock().Hash(),
def.Manifest.EphemeralGenesis,
```

```
vals...)
if err != nil {
return errors.Wrap(err, "make genesis")
}


// 5. 각 노드별 설정 파일 생성
for _, node := range def.Testnet.Nodes {
nodeDir := filepath.Join(def.Testnet.Dir, node.Name)
// CometBFT 설정
cfg, err := MakeConfig(def.Testnet, node, nodeDir)
if err != nil {
return err
}
// Halo 설정
if err := writeHaloConfig(def, nodeDir, logCfg, depCfg.testConfig, node, omniEV
return err
}
// 키 파일들
(privval.NewFilePV(node.PrivvalKey,
filepath.Join(nodeDir, PrivvalKeyFile),
filepath.Join(nodeDir, PrivvalStateFile),
)).Save()
}
```

```go
// 6. 보조 서비스 설정
if err := writeRelayerConfig(ctx, def, logCfg); err != nil {
return err
}
if err := writeMonitorConfig(ctx, def, logCfg, valPrivKeys); err != nil {
return err
}
if err := writeSolverConfig(ctx, def, logCfg); err != nil {
return err
}

// 7. 인프라 설정
if err := def.Infra.Setup(); err != nil {
return errors.Wrap(err, "setup provider")
}

return nil
}
```

3.2 환경 변수 설정

테스트 실행 시 다음 환경 변수들이 설정됩니다:

```go
const (
EnvInfraType = "INFRASTRUCTURE_TYPE"  // docker 또는 vmcompose
EnvInfraFile = "INFRASTRUCTURE_FILE"  // 인프라 데이터 파일
EnvE2EManifest = "E2E_MANIFEST"  // 매니페스트 파일 경로
EnvE2ENode = "E2E_NODE"  // 특정 노드 테스트용
EnvE2ERPCEndpoints = "E2E_RPC_ENDPOINTS"  // RPC 엔드포인트 파일
)
```

자동화된 검증 시스템

1. 테스트 실행 프레임워크

1.1 Go 테스트 실행

```go
func  Test(ctx context.Context, def Definition, cfg TestConfig) error {
// 1. 엔드포인트 파일 생성
endpoints := ExternalEndpoints(def)
endpointsFile := filepath.Join(networkDir, "endpoints.json")
if  endpointsBytes, err := json.Marshal(endpoints);  err != nil {
return  errors.Wrap(err, "marshaling endpoints")
} else  if  err := os.WriteFile(endpointsFile, endpointsBytes, 0644);  err != ni
return  errors.Wrap(err, "writing endpoints")
}
```

```go
// 2. 환경 변수 설정
manifestFile, err := filepath.Abs(def.Testnet.File)
if err != nil {
return errors.Wrap(err, "absolute manifest path")
}
os.Setenv(EnvE2EManifest, manifestFile)
os.Setenv(EnvInfraType, infd.Provider)
os.Setenv(EnvE2ERPCEndpoints, endpointsFile)

// 3. Go 테스트 실행
args := []string{"go", "test", "-timeout", "120s", "-count", "1"}
if cfg.Verbose {
args = append(args, "-v")
}
if cfg.RunArg != "" {
args = append(args, "-run", cfg.RunArg)
}
args = append(args, "github.com/omni-network/omni/e2e/test")

err = exec.CommandVerbose(ctx, args...)
if err != nil {
return errors.Wrap(err, "go tests failed")
}
```

```
return nil
}
```

1.2 테스트 타입별 실행

```
// 노드별 테스트
func testNode(t *testing.T, fn func(*testing.T, netconf.Network, *e2e.Node, []
test(t, testFunc{TestNode: fn})
}

// 포털별 테스트
func testPortal(t *testing.T, fn func(*testing.T, netconf.Network, Portal, []P
test(t, testFunc{TestPortal: fn})
}

// Omni EVM 테스트
func testOmniEVM(t *testing.T, fn func(*testing.T, ethclient.Client)) {
test(t, testFunc{TestOmniEVM: fn})
}

// 네트워크 전체 테스트
func testNetwork(t *testing.T, fn func(context.Context, *testing.T, NetworkDep
```

```
test(t, testFunc{TestNetwork: fn})

}
```

2. 핵심 검증 테스트

2.1 증명(Attestation) 검증

```
func TestApprovedAttestations(t *testing.T) {

testNode(t, func(t *testing.T, network netconf.Network, node *e2e.Node, portals

// 아카이브 노드만 테스트 (전체 상태 보유)

if node.Mode != types.ModeArchive {

return

}


client, err := node.Client()

require.NoError(t, err)

cprov := provider.NewABCI(client, network.ID)


// 각 포털 체인의 증명 검증

for _, portal := range portals {

for _, chainVer := range portal.Chain.ChainVersions() {

atts, err := fetchAllAtts(ctx, cprov, chainVer, node.StartAt > 0)

require.NoError(t, err)

// 비어있지 않은 블록들이 증명되었는지 확인
```

```
require.NotEmpty(t, atts, "No attestations for chain version: %v", chainVer)
}
}

// 합의 체인 증명 확인
consChain, ok := network.OmniConsensusChain()
require.True(t, ok)
for _, chainVer := range consChain.ChainVersions() {
atts, err := fetchAllAtts(ctx, cprov, chainVer, node.StartAt > 0)
require.NoError(t, err)
require.NotEmpty(t, atts)
}
})
}
```

2.2 검증자 업데이트 테스트

```
func TestApprovedValUpdates(t *testing.T) {
testNode(t, func(t *testing.T, network netconf.Network, node *e2e.Node, portals
// 검증자 업데이트가 있는 노드만 테스트
var hasUpdate bool
var power int64
for _, powers := range node.Testnet.ValidatorUpdates {
```

```go
for n, p := range powers {
if n.Name == node.Name {
hasUpdate = true
power = p
}
}
}

if !hasUpdate {
return
}

// 검증자 세트에서 업데이트 확인
valSetID := consChain.DeployHeight
for {
vals, ok, err := cprov.PortalValidatorSet(ctx, valSetID)
require.NoError(t, err)
require.True(t, ok, "Validator update not found")

for _, val := range vals {
if val.Address == addr && val.Power == power {
return // 검증자 업데이트 확인됨
}
```

```
}
valSetID++
}
})
}
```

2.3 브리지 기능 테스트

```
func  TestBridge(t *testing.T) {
maybeTestNetwork(t, skipFunc, func(ctx context.Context, t *testing.T, deps Netwo
// 브리지 배포
err := bridge.DeployBridge(ctx, deps.Testnet, deps.Backends)
require.NoError(t, err)

// 테스트 데이터 준비
toNatives := []bridge.ToAmount{
{To: addr1, Amount: bi.Ether(20)},
{To: addr2, Amount: bi.Ether(30)},
}
toL1s := []bridge.ToAmount{
{To: addr1, Amount: bi.Ether(2)},
{To: addr2, Amount: bi.Ether(3)},
}
```

```go
// 브리지 테스트 실행
err = bridge.Test(ctx, deps.Testnet, deps.Backends, toNatives, toL1s)
require.NoError(t, err)


// 잔액 검증
for _, test := range toNatives {
balance, err := omniBackend.BalanceAt(ctx, test.To, nil)
require.NoError(t, err)
require.Equal(t, balance, test.Amount)
}


// L1 브리지 잔액 검증
for _, test := range toL1s {
require.Eventually(t, func() bool {
balance, err := l1Token.BalanceOf(nil, test.To)
return err == nil && bi.EQ(balance, test.Amount)
}, time.Second*30, time.Second)
}
})
}
```

3. 대기 및 동기화 메커니즘

3.1 블록 높이 대기

```go
func Wait(ctx context.Context, testnet *e2e.Testnet, blocks int64) error {
block, _, err := waitForHeight(ctx, testnet, 0)
if err != nil {
return err
}

return WaitUntil(ctx, testnet, block.Height+blocks)
}

func WaitUntil(ctx context.Context, testnet *e2e.Testnet, height int64) error
log.Info(ctx, "Waiting for nodes to reach height", "height", height)
_, err := waitForAllNodes(ctx, testnet, height, waitingTime(len(testnet.Nodes),
if err != nil {
return err
}

return nil
}

// 대기 시간 추정 (노드 수와 높이에 따라)
func waitingTime(nodes int, height int64) time.Duration {
```

```
return time.Duration(20+(int64(nodes)*height)) * time.Second
}
```

3.2 크로스체인 메시지 제출 대기

```
func WaitAllSubmissions(ctx context.Context, network netconf.Network, portals
log.Info(ctx, "Waiting for submissions on all destination chains")
ctx, cancel := context.WithTimeout(ctx, 5*time.Minute)
defer cancel()

for _, dest := range portals {
for _, stream := range network.StreamsTo(dest.Chain.ChainID) {
if stream.SourceChainID == network.ID.Static().OmniConsensusChainIDUint64() {
continue
}

backoff := expbackoff.New(ctx, expbackoff.WithPeriodicConfig(time.Second))
for i := 0; ; i++ {
if ctx.Err() != nil {
return errors.Wrap(ctx.Err(), "timeout waiting for submissions")
}

// 소스와 목적지 오프셋 확인
srcOffset, err := portals[stream.SourceChainID].Contract.OutXMsgOffset(nil, dest
```

```
if  err != nil {
return  errors.Wrap(err, "get outXMsgOffset")
}

destOffset, err := dest.Contract.InXMsgOffset(nil, stream.SourceChainID, uint64(
if  err != nil {
return  errors.Wrap(err, "get inXmsgOffset")
}

// 최소 메시지 수 및 동기화 확인
if  srcOffset >= minimum && withinOne(srcOffset, destOffset) {
break
}

if  i%5 == 0 {
log.Debug(ctx, "Waiting for submissions on destination chain",
"stream", network.StreamName(stream),
"src_offset", srcOffset,
"dest_offset", destOffset,
"minimum", minimum,
)
}
```

```
backoff()
}
}
}

return nil
}
```

장애 시뮬레이션 (Perturbations)

1. 노드 레벨 장애 시뮬레이션

```
func perturbNode(ctx context.Context, node *e2e.Node, perturbation e2e.Perturba
switch perturbation {
case e2e.PerturbationDisconnect:
// 네트워크 연결 끊기/복구
networkName := testnet.Name + "_" + testnet.Name
if err := docker.Exec(ctx, "network", "disconnect", networkName, name); err !=
return nil, errors.Wrap(err, "disconnect node from network")
}
time.Sleep(10 * time.Second)
if err := docker.Exec(ctx, "network", "connect", networkName, name); err != ni
return nil, errors.Wrap(err, "connect node tp network")
```

```
}

case e2e.PerturbationKill:
// 프로세스 강제 종료/재시작
if err := docker.ExecCompose(ctx, testnet.Dir, "kill", "-s", "SIGKILL", name);
return nil, errors.Wrap(err, "kill node")
}
if err := docker.ExecCompose(ctx, testnet.Dir, "start", name); err != nil {
return nil, errors.Wrap(err, "start node")
}

case e2e.PerturbationPause:
// 프로세스 일시 정지/재개
if err := docker.ExecCompose(ctx, testnet.Dir, "pause", name); err != nil {
return nil, errors.Wrap(err, "pause node")
}
time.Sleep(10 * time.Second)
if err := docker.ExecCompose(ctx, testnet.Dir, "unpause", name); err != nil {
return nil, errors.Wrap(err, "unpause node")
}

case e2e.PerturbationRestart:
// 정상 재시작
```

```go
if err := docker.ExecCompose(ctx, testnet.Dir, "restart", name); err != nil {
return nil, errors.Wrap(err, "restart node")
}

case e2e.PerturbationUpgrade:
// 버전 업그레이드
if err := docker.ExecCompose(ctx, testnet.Dir, "down", name); err != nil {
return nil, errors.Wrap(err, "stop service")
}
if err := docker.ReplaceUpgradeImage(testnet.Dir, name); err != nil {
return nil, errors.Wrap(err, "upgrade service")
}
if err := docker.ExecCompose(ctx, testnet.Dir, "up", "-d", name); err != nil {
return nil, errors.Wrap(err, "start service")
}
}

// 복구 대기 및 상태 확인
status, err := waitForNode(ctx, node, 0, 20*time.Second)
if err != nil {
return nil, err
}
```

```
log.Info(ctx, "Node recovered from perturbation", "height", status.SyncInfo.Late
return status, nil
}
```

2. 서비스 레벨 장애 시뮬레이션

```
func perturbService(ctx context.Context, service string, testnetDir string, p
switch perturb {
case types.PerturbRestart:
if err := docker.ExecCompose(ctx, testnetDir, "restart", service); err != nil
return errors.Wrap(err, "restart service")
}

case types.PerturbStopStart:
if err := docker.ExecCompose(ctx, testnetDir, "stop", service); err != nil {
return errors.Wrap(err, "stop service")
}
time.Sleep(5 * time.Second)
if err := docker.ExecCompose(ctx, testnetDir, "start", service); err != nil {
return errors.Wrap(err, "start service")
}
```

```
case  types.PerturbRollback:
if  err := docker.ExecCompose(ctx, testnetDir, "stop", service);  err != nil {
return  errors.Wrap(err, "stop service")
}
if  err := docker.ExecCompose(ctx, testnetDir, "run", service, "run", "rollback"
return  errors.Wrap(err, "rollback service")
}
if  err := docker.ExecCompose(ctx, testnetDir, "start", service);  err != nil {
return  errors.Wrap(err, "start service")
}


case  types.PerturbFuzzyHeadAttRoot, types.PerturbFuzzyHeadDropBlocks, types.Per
// FuzzyHead 테스트 - 일시적 오동작 시뮬레이션
if  err := docker.ExecCompose(ctx, testnetDir, "exec", service, "wget", "-O-", "
return  errors.Wrap(err, "enable fuzzy head")
}
time.Sleep(6 * time.Second)
if  err := docker.ExecCompose(ctx, testnetDir, "exec", service, "wget", "-O-", "
return  errors.Wrap(err, "disable fuzzy head")
}


case  types.PerturbUpgrade:
if  err := docker.ExecCompose(ctx, testnetDir, "down", service);  err != nil {
```

```
return errors.Wrap(err, "down service")
}
if err := docker.ReplaceUpgradeImage(testnetDir, service); err != nil {
return errors.Wrap(err, "upgrade service")
}
if err := docker.ExecCompose(ctx, testnetDir, "up", "-d", service); err != nil
return errors.Wrap(err, "up service")
}
}

return nil
}
```

3. FuzzyHead 테스트

FuzzyHead는 EVM 엔진의 일시적 오동작을 시뮬레이션하는 특수 테스트입니다:

- **PerturbFuzzyHeadDropBlocks**: 블록 드롭 시뮬레이션
- **PerturbFuzzyHeadDropMsgs**: 메시지 드롭 시뮬레이션
- **PerturbFuzzyHeadAttRoot**: 증명 루트 불일치 시뮬레이션
- **PerturbFuzzyHeadMoreMsgs**: 중복 메시지 시뮬레이션

실시간 모니터링 시스템

1. 영수증 모니터링

```go
func StartMonitoringReceipts(ctx context.Context, def Definition) func() error
// 모든 트랜잭션 영수증 상태 추적
// 실패한 트랜잭션 감지 및 보고
// 가스 사용량 및 수수료 모니터링
}
```

2. 검증자 업데이트 모니터링

```go
func StartValidatorUpdates(ctx context.Context, def Definition) func() error {
// 검증자 세트 변경 추적
// 스테이킹 변경 모니터링
// 슬래싱 이벤트 감지
}
```

3. 포털 추가 모니터링

```go
func startAddingMockPortals(ctx context.Context, def Definition) func() error {
// 새로운 포털 등록 추적
// 포털 상태 변경 모니터링
// 크로스체인 연결 상태 확인
}
```

최종 검증 및 정리

1. 종합 검증 과정

```
func finalValidation(ctx context.Context, def Definition, pingpong *pingpong.XD

// 1. 모든 크로스체인 메시지 전송 완료 대기

log.Info(ctx, "Waiting for all cross chain messages to be sent")

select {

case <-ctx.Done():

return errors.Wrap(ctx.Err(), "cancel")

case err := <-msgsErr:

if err != nil {

return err

}

}


// 2. 모든 메시지 제출 완료 대기

network := networkFromDef(def)

if err := WaitAllSubmissions(ctx, network, def.Netman().Portals(), sum(msgBatch

return err

}


// 3. PingPong 잔액 확인 및 완료 대기
```

```go
if err := pingpong.LogBalances(ctx); err != nil {
return err
}
if err := pingpong.WaitDone(ctx); err != nil {
return errors.Wrap(err, "wait for pingpong")
}

// 4. 모니터링 중지 및 결과 수집
if err := stopReceiptMonitor(); err != nil {
return errors.Wrap(err, "stop deploy")
}
if err := stopValidatorUpdates(); err != nil {
return errors.Wrap(err, "stop validator updates")
}

// 5. 메트릭 로깅
if err := LogMetrics(ctx, def); err != nil {
return err
}

// 6. 패닉 감지
if err := detectPanics(ctx, def); err != nil {
return err
```

```
}

return nil
}
```

2. 메트릭 수집 및 로깅

```
func LogMetrics(ctx context.Context, def Definition) error {
// 네트워크 성능 메트릭 수집
// 트랜잭션 처리량 통계
// 크로스체인 메시지 지연시간
// 리소스 사용량 (CPU, 메모리, 디스크)
// 에러율 및 성공률
}
```

3. 패닉 및 에러 감지

```
func detectPanics(ctx context.Context, def Definition) error {
// 로그에서 패닉 메시지 검색
// 예상치 못한 종료 감지
// 메모리 누수 확인
// 데드락 감지
```

```
}
```

CLI 명령어 인터페이스

1. 주요 명령어

```
# 전체 E2E 테스트 실행
e2e -f e2e/manifests/devnet0.toml

# 개별 단계 실행
e2e -f manifest.toml setup # 설정 생성
e2e -f manifest.toml deploy # 네트워크 배포
e2e -f manifest.toml test # 테스트만 실행
e2e -f manifest.toml logs # 로그 출력
e2e -f manifest.toml clean # 정리

# 특정 노드만 테스트
E2E_NODE=validator01 e2e -f manifest.toml test

# 특정 테스트만 실행
e2e -f manifest.toml test -run TestBridge
```

2. 설정 플래그

```go
type E2ETestConfig struct {
Preserve  bool  // 테스트 후 컨테이너 보존
}

type DeployConfig struct {
PingPongN  uint64  // PingPong 홉 수
PingPongP  uint64  // 병렬 PingPong 수
PingPongL  uint64  // Latest 확인 레벨 사용 수
}

type TestConfig struct {
Verbose  bool  // 상세 출력
RunArg  string  // 실행할 테스트 패턴
SkipArg  string  // 건너뛸 테스트 패턴
}
```

정리

E2E 테스트 실행 및 검증 과정은:

1. **체계적인 단계별 실행**: 설정 → 배포 → 테스트 → 검증 → 정리
2. **포괄적인 검증**: 합의, 크로스체인 메시징, 브리지, 검증자 업데이트 등 모든 기능 테스트
3. **실시간 모니터링**: 영수증, 검증자 상태, 포털 상태 등 실시간 추적
4. **장애 복원력 테스트**: 다양한 perturbation을 통한 네트워크 안정성 검증

5. **자동화된 검증**: Go 테스트 프레임워크를 통한 자동화된 검증 로직

6. **유연한 실행 옵션**: CLI를 통한 다양한 실행 모드 지원

이 시스템을 통해 Omni 프로토콜의 모든 측면이 실제 운영 환경과 유사한 조건에서 철저히 테스트되고 검증됩니다.